

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 9”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 21 Mei 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN.....	3
2.1 Langkah Kerja.....	3
2.2 Shellsort_2511533012.....	3
2.3 Quicksort_2511533012	5
2.4 BubblesortGUI_2511533012	7
2.5 MergesortGUI_2511533012	9
BAB III KESIMPULAN	12
3.1 Ringkasan.....	12
3.2 Kesimpulan.....	12
DAFTAR PUSTAKA	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu komponen penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara terorganisir. Pada praktikum ini, mahasiswa mempelajari implementasi **Binary Tree** dan **Graph** menggunakan bahasa pemrograman Java. Binary Tree digunakan untuk menyimpan data dalam bentuk hierarki yang terdiri dari root, parent, dan child, sedangkan Graph digunakan untuk merepresentasikan hubungan antar simpul yang saling terhubung.

Melalui praktikum ini, mahasiswa mempelajari cara membangun struktur pohon biner, melakukan traversal pada pohon, menghitung jumlah simpul, serta melakukan penelusuran graph menggunakan metode **Depth First Search (DFS)** dan **Breadth First Search (BFS)**. Pemahaman terhadap struktur data ini sangat penting karena banyak digunakan dalam pengembangan perangkat lunak dan pengolahan data.

1.2 Tujuan

Praktikum ini bertujuan untuk memahami konsep Binary Tree dan Graph serta cara mengimplementasikannya menggunakan Java. Selain itu, praktikum dilakukan agar mahasiswa mampu melakukan traversal pada pohon biner, menghitung jumlah simpul, serta melakukan penelusuran graph menggunakan metode DFS dan BFS.

1.3 Manfaat

Melalui praktikum ini, mahasiswa dapat memahami cara kerja struktur data non-linear seperti Binary Tree dan Graph. Praktikum juga membantu meningkatkan kemampuan dalam mengelola data yang memiliki hubungan

hierarkis maupun hubungan antar simpul serta memberikan dasar yang kuat untuk mempelajari algoritma dan struktur data yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan9_2511533012**.
3. Menambahkan tiga buah file Java, yaitu Node_2511533012.java, BTree_2511533012.java, BtreeDriver_2511533012.java, GraphTraversal_2511533012.java.
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 Node_2511533012

```
package pekan9_2511533012;

public class Node_2511533012 {
    int data_3012;
    Node_2511533012 left_3012;
    Node_2511533012 right_3012;
    public Node_2511533012(int data_3012) {
        this.data_3012 = data_3012;
        left_3012 = null;
        right_3012 = null;
    }
    public void setLeft_2511533012(Node_2511533012 node_3012) {
        if (left_3012 == null)
            left_3012 = node_3012;
    }
}
```

```

    public void setRight_2511533012(Node_2511533012 node_3012) {
        if (right_3012 == null)
            right_3012 = node_3012;
    }
    public Node_2511533012 getLeft_2511533012() {
        return left_3012;
    }
    public Node_2511533012 getRight_2511533012() {
        return right_3012;
    }
    public int getData_2511533012() {
        return data_3012;
    }
    public void setData_2511533012(int data_3012) {
        this.data_3012 = data_3012;
    }

    void printPreorder_2511533012(Node_2511533012 node_3012) {
        if (node_3012 == null)
            return;
        System.out.print(node_3012.data_3012 + " ");
        printPreorder_2511533012(node_3012.left_3012);
        printPreorder_2511533012(node_3012.right_3012);
    }
    void printPostorder_2511533012(Node_2511533012 node_3012) {
        if (node_3012 == null)
            return;
        printPostorder_2511533012(node_3012.left_3012);
        printPostorder_2511533012(node_3012.right_3012);
        System.out.print(node_3012.data_3012 + " ");
    }
    void printInorder_2511533012(Node_2511533012 node_3012) {
        if (node_3012 == null)
            return;
        printInorder_2511533012(node_3012.left_3012);
        System.out.print(node_3012.data_3012 + " ");
        printInorder_2511533012(node_3012.right_3012);
    }
    public String print_2511533012() {
        return this.print_2511533012("", true, "");
    }
    public String print_2511533012(String prefix, boolean isTail,
String sb) {
        if (right_3012 != null) {
            right_3012.print_2511533012(prefix + (isTail ? "| " : "
"), false, sb);
        }
        System.out.println(prefix + (isTail ? "\\-- " : "/-- ") +
data_3012);
        if (left_3012 != null) {
            left_3012.print_2511533012(prefix + (isTail ? " " : "|
"), true, sb);
        }
    }

```

```

        return sb;
    }
}

```

Analisis:

Program ini merupakan class dasar yang digunakan untuk membentuk node pada struktur **Binary Tree**.

Class ini menyimpan data serta memiliki referensi ke child kiri dan child kanan. Selain itu, terdapat method untuk mengatur node kiri dan kanan, mengambil nilai data, serta melakukan traversal preorder, inorder, dan postorder.

Kesimpulan: Program ini berfungsi sebagai dasar pembentukan struktur Binary Tree

2.3 Btree_2511533012

```

package pekan9_2511533012;

public class BTree_2511533012 {
    private Node_2511533012 root_3012;
    private Node_2511533012 currentNode_3012;
    public BTree_2511533012() {
        root_3012 = null;
    }
    public boolean search_2511533012(int data_3012) {
        return search_2511533012(root_3012, data_3012);
    }
    private boolean search_2511533012(Node_2511533012 node_3012, int data_3012) {
        if (node_3012.getData_2511533012() == data_3012)
            return true;
        if (node_3012.getLeft_2511533012() != null)
            if (search_2511533012(node_3012.getLeft_2511533012(), data_3012))
                return true;
        if (node_3012.getRight_2511533012() != null)
            if (search_2511533012(node_3012.getRight_2511533012(), data_3012))
                return true;
        return false;
    }
    public void printInorder_2511533012() {
        root_3012.printInorder_2511533012(root_3012);
    }
}

```



```

    public void printPreorder_2511533012() {
        root_3012.printPreorder_2511533012(root_3012);
    }
    public void printPostorder_2511533012() {
        root_3012.printPostorder_2511533012(root_3012);
    }

    public Node_2511533012 getRoot_2511533012() {
        return root_3012;
    }
    public boolean isEmpty_2511533012() {
        return root_3012 == null;
    }
    public int countNode_2511533012() {
        return countNode_2511533012(root_3012);
    }
    private int countNode_2511533012(Node_2511533012 node_3012) {
        int count_3012 = 1;
        if (node_3012 == null) {
            return 0;
        } else {
            count_3012 +=
countNode_2511533012(node_3012.getLeft_2511533012());
            count_3012 +=
countNode_2511533012(node_3012.getRight_2511533012());
            return count_3012;
        }
    }
    public void print_2511533012() {
        root_3012.print_2511533012();
    }
    public Node_2511533012 getCurrent_2511533012() {
        return currentNode_3012;
    }
    public void setCurrent_2511533012(Node_2511533012 node_3012) {
        currentNode_3012 = node_3012;
    }
    public void setRoot_2511533012(Node_2511533012 root_3012) {
        this.root_3012 = root_3012;
    }
}

```

Analisis :

Program ini digunakan untuk mengelola struktur **Binary Tree**.

Class menyediakan beberapa fungsi seperti pencarian data pada pohon, menghitung jumlah simpul, mengecek apakah pohon kosong, serta menampilkan traversal inorder, preorder, dan postorder. Program juga dapat menampilkan struktur pohon secara visual.

Kesimpulan: Program ini berfungsi sebagai pengelola utama operasi pada Binary Tree.

2.4 BtreeDriver_2511533012

```
package pekan9_2511533012;

public class BtreeDriver_2511533012 {
    public static void main(String[] args) {
        BTree_2511533012 tree = new BTree_2511533012();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree.countNode_2511533012());

        Node_2511533012 root_3012 = new Node_2511533012(1);

        tree.setRoot_2511533012(root_3012);
        System.out.println("Jumlah simpul jika hanya ada root: ");
        System.out.println(tree.countNode_2511533012());
        Node_2511533012 node2_3012 = new Node_2511533012(2);
        Node_2511533012 node3_3012 = new Node_2511533012(3);
        Node_2511533012 node4_3012 = new Node_2511533012(4);
        Node_2511533012 node5_3012 = new Node_2511533012(5);
        Node_2511533012 node6_3012 = new Node_2511533012(6);
        Node_2511533012 node7_3012 = new Node_2511533012(7);
        Node_2511533012 node8_3012 = new Node_2511533012(8);
        Node_2511533012 node9_3012 = new Node_2511533012(9);
        root_3012.setLeft_2511533012(node2_3012);
        node2_3012.setLeft_2511533012(node4_3012);
        node2_3012.setRight_2511533012(node5_3012);
        node4_3012.setRight_2511533012(node8_3012);
        root_3012.setRight_2511533012(node3_3012);
        node3_3012.setLeft_2511533012(node6_3012);
        node3_3012.setRight_2511533012(node7_3012);
        node6_3012.setLeft_2511533012(node9_3012);

        tree.setCurrent_2511533012(tree.getRoot_2511533012());
        System.out.println("menampilkan simpul terakhir: ");
```

```

System.out.println(tree.getCurrent_2511533012().getData_2511533012()
);
    System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
    System.out.println(tree.countNode_2511533012());
    System.out.println("InOrder: ");
    tree.printInorder_2511533012();
    System.out.println("\nPreOrder: ");
    tree.printPreorder_2511533012();
    System.out.println("\nPostOrder: ");
    tree.printPostorder_2511533012();
    System.out.println("\nDmenampilkan simpul dalam bentuk
pohon");
    tree.print_2511533012();
}
}

```

Output :

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root:
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|      /-- 7
|  /-- 3
|  |  \-- 6
|  |  \-- 9
|  |  \-- 1
|  |  /-- 5
|  |  \-- 2
|  |  |  /-- 8
|  |  |  \-- 4

```

Analisis :

Program ini berfungsi sebagai driver atau program utama untuk menguji class Binary Tree.

Program membuat beberapa node, menyusunnya menjadi sebuah pohon biner, kemudian menghitung jumlah simpul yang ada. Selain itu, program juga

menampilkan hasil traversal inorder, preorder, dan postorder serta menampilkan bentuk pohon yang telah dibuat.

Kesimpulan: Program ini digunakan untuk menguji dan menampilkan hasil implementasi Binary Tree.

2.5 GraphTraversal_2511533012

```
package pekan9_2511533012;
import java.util.*;

public class GraphTraversal_2511533012 {
    private Map<String, List<String>> graph_3012 = new HashMap<>();

    public void addEdge_2511533012(String node1_3012, String
node2_3012) {
        graph_3012.putIfAbsent(node1_3012, new ArrayList<>());
        graph_3012.putIfAbsent(node2_3012, new ArrayList<>());
        graph_3012.get(node1_3012).add(node2_3012);
        graph_3012.get(node2_3012).add(node1_3012);
    }

    public void printGraph_2511533012() {
        System.out.println("Graf awal (adjacency list): ");
        for (String node_3012 : graph_3012.keySet()) {
            System.out.print(node_3012 + " -> ");
            List<String> neighbors_3012 = graph_3012.get(node_3012);
            System.out.println(String.join(", ", neighbors_3012));
        }
        System.out.println();
    }

    public void dfs_2511533012(String start_3012) {
        Set<String> visited_3012 = new HashSet<>();
        System.out.println("Penelusuran DFS: ");
        dfsHelper_2511533012(start_3012, visited_3012);
        System.out.println();
    }

    private void dfsHelper_2511533012(String current_3012,
Set<String> visited_3012) {
        if (visited_3012.contains(current_3012))
            return;
        visited_3012.add(current_3012);
        System.out.print(current_3012 + " ");
        for (String neighbor_3012 :
graph_3012.getDefault(current_3012, new ArrayList<>())) {
            dfsHelper_2511533012(neighbor_3012, visited_3012);
        }
    }

    public void bfs_2511533012(String start_3012) {
        Set<String> visited_3012 = new HashSet<>();
        Queue<String> queue_3012 = new LinkedList<>();
```

```

        queue_3012.add(start_3012);
        visited_3012.add(start_3012);
        System.out.println("Penelusuran BFS: ");
        while (!queue_3012.isEmpty()) {
            String current_3012 = queue_3012.poll();
            System.out.print(current_3012 + " ");
            for (String neighbor_3012 :
graph_3012.getDefault(current_3012, new ArrayList<>())) {
                if (!visited_3012.contains(neighbor_3012)) {
                    queue_3012.add(neighbor_3012);
                    visited_3012.add(neighbor_3012);
                }
            }
        }
        System.out.println();
    }
    public static void main(String[] args) {
        GraphTraversal_2511533012 graph_3012 = new
GraphTraversal_2511533012();

        graph_3012.addEdge_2511533012("A", "B");
        graph_3012.addEdge_2511533012("A", "C");
        graph_3012.addEdge_2511533012("B", "D");
        graph_3012.addEdge_2511533012("B", "E");

        System.out.println("Graf awal adalah: ");
        graph_3012.printGraph_2511533012();

        graph_3012.dfs_2511533012("A");
        graph_3012.bfs_2511533012("A");
    }
}

```

Output :

```

<terminated> GraphTraversal_2511533012.java Applica
Graf awal adalah:
Graf awal (adjacency list):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

```

Analisis :

Program ini mengimplementasikan struktur data **Graph** menggunakan adjacency list.

Program menyediakan method untuk menambahkan hubungan antar simpul, menampilkan graph, serta melakukan penelusuran menggunakan metode DFS dan BFS. Data graph disimpan menggunakan HashMap dan ArrayList sehingga hubungan antar simpul dapat dikelola dengan mudah.

Kesimpulan: Program ini menunjukkan penerapan traversal graph menggunakan metode DFS dan BFS.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-9, mahasiswa mempelajari implementasi struktur data Binary Tree dan Graph menggunakan Java. Praktikum mencakup pembuatan node, traversal pohon menggunakan preorder, inorder, dan postorder, serta penelusuran graph menggunakan metode DFS dan BFS. Melalui praktikum ini, mahasiswa memahami bagaimana data dapat disimpan dan ditelusuri dalam struktur non-linear.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang sangat penting dalam pemrograman. Binary Tree digunakan untuk menyimpan data secara hierarkis dan mendukung berbagai metode traversal, sedangkan Graph digunakan untuk merepresentasikan hubungan antar simpul dan dapat ditelusuri menggunakan DFS maupun BFS. Praktikum ini membantu mahasiswa memahami konsep dasar struktur data non-linear sebagai bekal untuk mempelajari algoritma yang lebih lanjut.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. PetaKampus_2511533012.java

```
package pekan9_2511533012;

import java.awt.*;
import java.util.*;
import javax.swing.*;

public class PetaKampus_2511533012 extends JFrame {

    private Map<String, java.util.List<String>> graph_3012 = new
    LinkedHashMap<>();
    private Map<String, Point> posisi_3012 = new HashMap<>();
    private Set<String> visited_3012 = new HashSet<>();
    private java.util.List<String> path_3012 = new ArrayList<>();

    private JComboBox<String> startBox_3012;
    private JComboBox<String> goalBox_3012;
    private JTextArea hasilArea_3012;
    private GraphPanel_3012 panelGraph_3012;

    public PetaKampus_2511533012() {
        setTitle("Pencarian Jalur BFS dan DFS - 2511533012");
        setSize(950, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        buatGraph_3012();

        JPanel topPanel_3012 = new JPanel();
        topPanel_3012.add(new JLabel("Lokasi Awal:"));
        startBox_3012 = new JComboBox<>(graph_3012.keySet().toArray(new
String[0]));
        topPanel_3012.add(startBox_3012);

        topPanel_3012.add(new JLabel("Lokasi Tujuan:"));
        goalBox_3012 = new JComboBox<>(graph_3012.keySet().toArray(new
String[0]));
        topPanel_3012.add(goalBox_3012);

        JButton bfsButton_3012 = new JButton("BFS");
        JButton dfsButton_3012 = new JButton("DFS");
        JButton resetButton_3012 = new JButton("RESET");

        topPanel_3012.add(bfsButton_3012);
        topPanel_3012.add(dfsButton_3012);
        topPanel_3012.add(resetButton_3012);

        panelGraph_3012 = new GraphPanel_3012();
```

```

        hasilArea_3012 = new JTextArea(7, 30);
        hasilArea_3012.setEditable(false);
        hasilArea_3012.setFont(new Font("Monospaced", Font.PLAIN, 14));

        add(topPanel_3012, BorderLayout.NORTH);
        add(panelGraph_3012, BorderLayout.CENTER);
        add(new JScrollPane(hasilArea_3012), BorderLayout.SOUTH);

        bfsButton_3012.addActionListener(e -> BFS_3012());
        dfsButton_3012.addActionListener(e -> DFS_3012());
        resetButton_3012.addActionListener(e -> resetGraph_3012());
    }

    private void buatGraph_3012() {
        String[] lokasi_3012 = {
            "Gerbang Utama", "Rektorat", "Perpustakaan", "Fakultas
Teknik",
            "Fakultas Ekonomi", "Masjid Kampus", "Kantin", "Labor
Komputer",
            "Aula", "Parkiran", "Gedung Kuliah", "Lapangan"
        };

        for (String lokasi : lokasi_3012) {
            graph_3012.put(lokasi, new ArrayList<>());
        }

        tambahEdge_3012("Gerbang Utama", "Rektorat");
        tambahEdge_3012("Gerbang Utama", "Parkiran");
        tambahEdge_3012("Rektorat", "Perpustakaan");
        tambahEdge_3012("Rektorat", "Fakultas Ekonomi");
        tambahEdge_3012("Perpustakaan", "Fakultas Teknik");
        tambahEdge_3012("Perpustakaan", "Gedung Kuliah");
        tambahEdge_3012("Fakultas Teknik", "Labor Komputer");
        tambahEdge_3012("Fakultas Teknik", "Aula");
        tambahEdge_3012("Fakultas Ekonomi", "Kantin");
        tambahEdge_3012("Fakultas Ekonomi", "Masjid Kampus");
        tambahEdge_3012("Masjid Kampus", "Kantin");
        tambahEdge_3012("Kantin", "Lapangan");
        tambahEdge_3012("Labor Komputer", "Gedung Kuliah");
        tambahEdge_3012("Aula", "Lapangan");
        tambahEdge_3012("Parkiran", "Masjid Kampus");
        tambahEdge_3012("Parkiran", "Lapangan");

        posisi_3012.put("Gerbang Utama", new Point(80, 250));
        posisi_3012.put("Rektorat", new Point(220, 150));
        posisi_3012.put("Perpustakaan", new Point(400, 90));
        posisi_3012.put("Fakultas Teknik", new Point(620, 90));
        posisi_3012.put("Fakultas Ekonomi", new Point(360, 250));
        posisi_3012.put("Masjid Kampus", new Point(250, 390));
        posisi_3012.put("Kantin", new Point(480, 370));
        posisi_3012.put("Labor Komputer", new Point(750, 170));
        posisi_3012.put("Aula", new Point(730, 300));
        posisi_3012.put("Parkiran", new Point(120, 430));
    }

```

```

posisi_3012.put("Gedung Kuliah", new Point(570, 220));
posisi_3012.put("Lapangan", new Point(650, 430));
}

private void tambahEdge_3012(String a_3012, String b_3012) {
    graph_3012.get(a_3012).add(b_3012);
    graph_3012.get(b_3012).add(a_3012);
}

public void BFS_3012() {
    resetData_3012();

    String start_3012 = (String) startBox_3012.getSelectedItem();
    String goal_3012 = (String) goalBox_3012.getSelectedItem();

    Queue<String> queue_3012 = new LinkedList<>();
    Map<String, String> parent_3012 = new HashMap<>();

    queue_3012.add(start_3012);
    visited_3012.add(start_3012);
    parent_3012.put(start_3012, null);

    while (!queue_3012.isEmpty()) {
        String current_3012 = queue_3012.poll();

        if (current_3012.equals(goal_3012)) {
            break;
        }

        for (String neighbor_3012 : graph_3012.get(current_3012)) {
            if (!visited_3012.contains(neighbor_3012)) {
                visited_3012.add(neighbor_3012);
                parent_3012.put(neighbor_3012, current_3012);
                queue_3012.add(neighbor_3012);
            }
        }
    }

    buatPath_3012(parent_3012, start_3012, goal_3012);
    displayPath_3012("BFS");
}

public void DFS_3012() {
    resetData_3012();

    String start_3012 = (String) startBox_3012.getSelectedItem();
    String goal_3012 = (String) goalBox_3012.getSelectedItem();

    Stack<String> stack_3012 = new Stack<>();
    Map<String, String> parent_3012 = new HashMap<>();

    stack_3012.push(start_3012);
    parent_3012.put(start_3012, null);
}

```

```

        while (!stack_3012.isEmpty()) {
            String current_3012 = stack_3012.pop();

            if (!visited_3012.contains(current_3012)) {
                visited_3012.add(current_3012);

                if (current_3012.equals(goal_3012)) {
                    break;
                }

                for (String neighbor_3012 :
graph_3012.get(current_3012)) {
                    if (!visited_3012.contains(neighbor_3012)) {
                        stack_3012.push(neighbor_3012);
                        if (!parent_3012.containsKey(neighbor_3012)) {
                            parent_3012.put(neighbor_3012,
current_3012);
                        }
                    }
                }
            }
        }

        buatPath_3012(parent_3012, start_3012, goal_3012);
        displayPath_3012("DFS");
    }

    private void buatPath_3012(Map<String, String> parent_3012, String
start_3012, String goal_3012) {
        path_3012.clear();

        if (!parent_3012.containsKey(goal_3012)) {
            return;
        }

        String current_3012 = goal_3012;
        while (current_3012 != null) {
            path_3012.add(current_3012);
            current_3012 = parent_3012.get(current_3012);
        }

        Collections.reverse(path_3012);
    }

    public void displayPath_3012(String metode_3012) {
        hasilArea_3012.setText("");
        hasilArea_3012.append("Hasil Pencarian Menggunakan " +
metode_3012 + "\n");

        if (path_3012.isEmpty()) {
            hasilArea_3012.append("Jalur: Tidak ditemukan\n");
        } else {

```

```

        hasilArea_3012.append("Jalur: " + String.join(" -> ",
path_3012) + "\n");
    }

    hasilArea_3012.append("Node Dikunjungi: " + visited_3012 +
"\n");
    hasilArea_3012.append("Jumlah Node Dieksplorasi: " +
visited_3012.size() + "\n");

    displayGraph_3012();
}

public void displayGraph_3012() {
    panelGraph_3012.repaint();
}

public void resetGraph_3012() {
    resetData_3012();
    hasilArea_3012.setText("Graph dikembalikan ke kondisi
awal.\n");
    displayGraph_3012();
}

private void resetData_3012() {
    visited_3012.clear();
    path_3012.clear();
}

class GraphPanel_3012 extends JPanel {
    protected void paintComponent(Graphics g_3012) {
        super.paintComponent(g_3012);
        setBackground(Color.WHITE);

        Graphics2D g2_3012 = (Graphics2D) g_3012;
        g2_3012.setStroke(new BasicStroke(2));

        for (String node_3012 : graph_3012.keySet()) {
            Point p1_3012 = posisi_3012.get(node_3012);

            for (String tetangga_3012 : graph_3012.get(node_3012))
            {
                Point p2_3012 = posisi_3012.get(tetangga_3012);
                g2_3012.setColor(Color.GRAY);
                g2_3012.drawLine(p1_3012.x, p1_3012.y, p2_3012.x,
p2_3012.y);
            }
        }

        for (String node_3012 : graph_3012.keySet()) {
            Point p_3012 = posisi_3012.get(node_3012);

            if (path_3012.contains(node_3012)) {
                g2_3012.setColor(Color.ORANGE);
            }
        }
    }
}

```

```

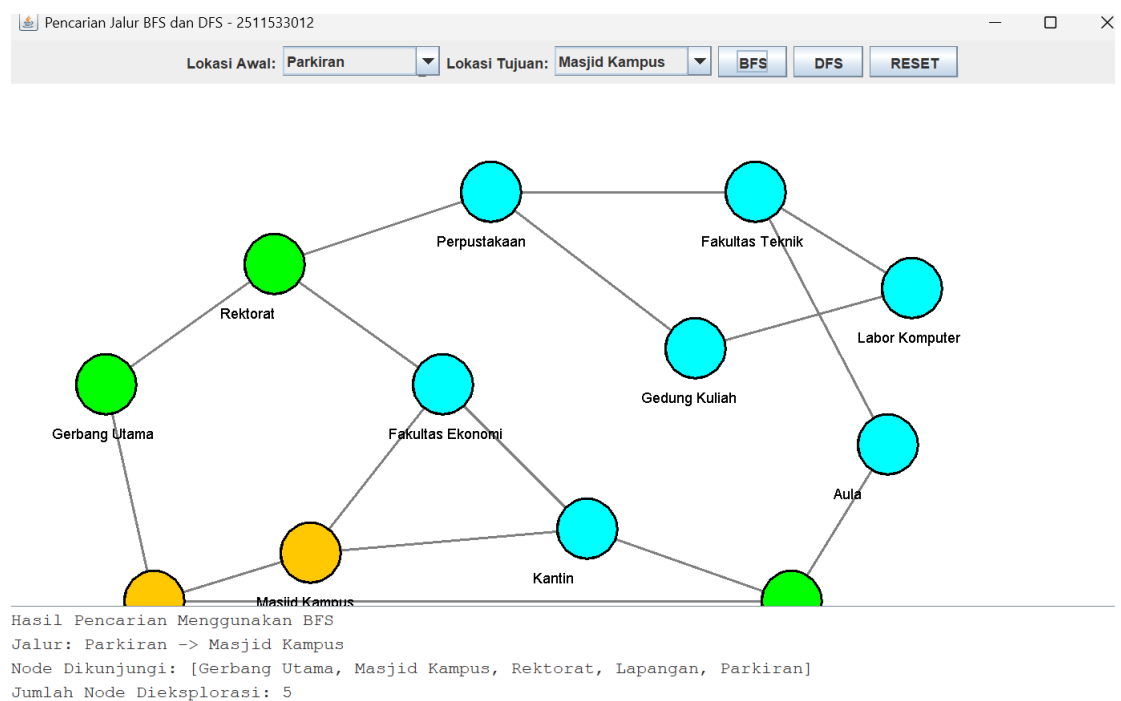
    } else if (visited_3012.contains(node_3012)) {
        g2_3012.setColor(Color.GREEN);
    } else {
        g2_3012.setColor(Color.CYAN);
    }

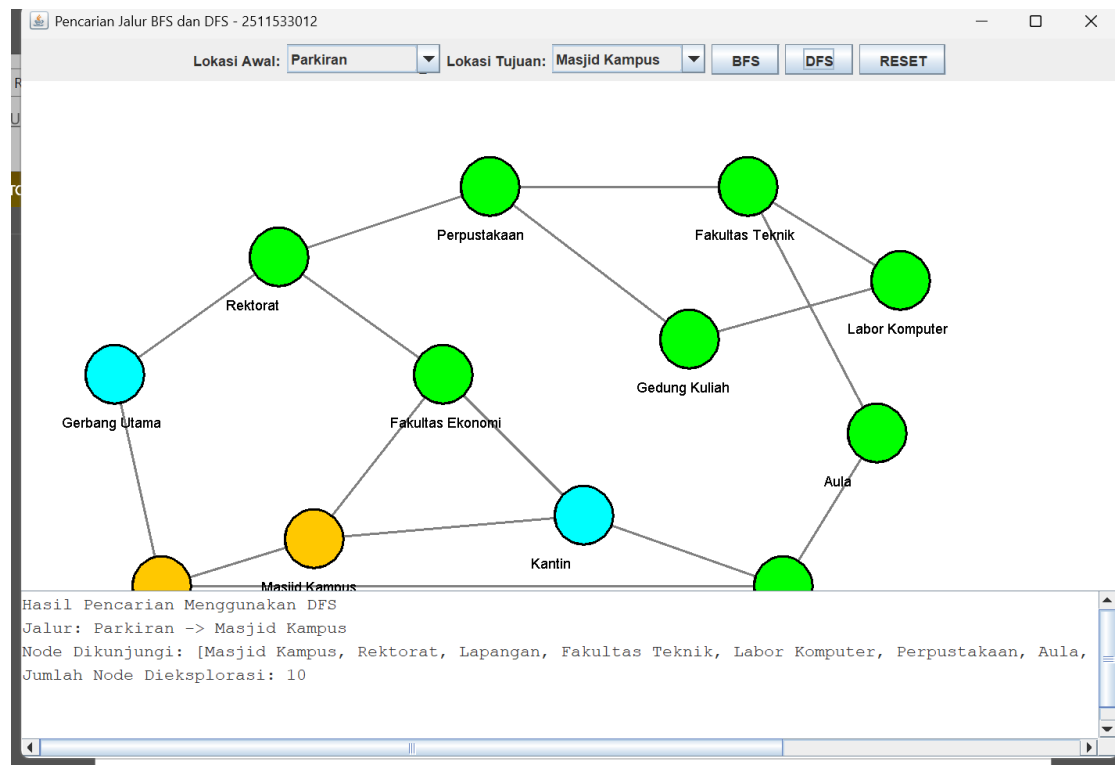
    g2_3012.fillOval(p_3012.x - 25, p_3012.y - 25, 50, 50);
    g2_3012.setColor(Color.BLACK);
    g2_3012.drawOval(p_3012.x - 25, p_3012.y - 25, 50, 50);
    g2_3012.drawString(node_3012, p_3012.x - 45, p_3012.y +
45);
    }
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new PetaKampus_2511533012().setVisible(true);
    });
}
}

```

Outpun Syntax program





Penjelasan:

Program ini dibuat untuk mensimulasikan pencarian jalur pada lingkungan kampus menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Lokasi-lokasi di dalam kampus direpresentasikan sebagai node atau simpul, sedangkan hubungan antar lokasi direpresentasikan sebagai edge atau sisi dalam sebuah graph. Program menyediakan antarmuka GUI yang memungkinkan pengguna memilih lokasi awal dan lokasi tujuan, kemudian menjalankan pencarian menggunakan BFS atau DFS.

Graph yang digunakan terdiri dari 12 lokasi yaitu Gerbang Utama, Rektorat, Perpustakaan, Fakultas Teknik, Fakultas Ekonomi, Masjid Kampus, Kantin, Labor Komputer, Aula, Parkiran, Gedung Kuliah, dan Lapangan. Setiap lokasi saling terhubung sesuai dengan jalur yang telah ditentukan sehingga membentuk sebuah graph yang dapat digunakan untuk proses pencarian.

Ketika tombol BFS ditekan, program akan menjalankan algoritma Breadth First Search. Algoritma ini bekerja dengan cara mengunjungi node

secara melebar atau per tingkat. Pencarian dimulai dari node awal yang dipilih pengguna, kemudian seluruh tetangga terdekat akan dikunjungi terlebih dahulu sebelum melanjutkan ke tingkat berikutnya. BFS menggunakan struktur data Queue (antrian) untuk mengatur urutan node yang akan dikunjungi. Kelebihan BFS adalah mampu menemukan jalur terpendek dari lokasi awal menuju lokasi tujuan pada graph yang tidak memiliki bobot.

Ketika tombol DFS ditekan, program akan menjalankan algoritma Depth First Search. Algoritma ini bekerja dengan cara menelusuri node sedalam mungkin sebelum kembali ke node sebelumnya. DFS menggunakan struktur data Stack (tumpukan) untuk menyimpan node yang akan dikunjungi. Pencarian akan terus dilakukan hingga mencapai tujuan atau tidak ada lagi node yang dapat dieksplorasi. Berbeda dengan BFS, DFS tidak selalu menghasilkan jalur terpendek, tetapi biasanya membutuhkan memori yang lebih sedikit.

Selama proses pencarian, program menyimpan informasi parent atau node asal dari setiap node yang dikunjungi. Informasi tersebut digunakan untuk membentuk kembali jalur dari lokasi tujuan menuju lokasi awal setelah pencarian selesai. Jalur yang ditemukan kemudian ditampilkan pada area hasil pencarian sehingga pengguna dapat melihat rute yang dilalui oleh algoritma.

Program juga menampilkan urutan node yang telah dikunjungi selama proses pencarian beserta jumlah node yang dieksplorasi. Informasi ini dapat digunakan untuk membandingkan performa BFS dan DFS dalam menemukan jalur pada graph yang sama.

Visualisasi graph ditampilkan menggunakan panel khusus yang menggambarkan setiap node dalam bentuk lingkaran dan setiap edge dalam bentuk garis penghubung. Warna node digunakan untuk menunjukkan status pencarian. Node yang belum dikunjungi ditampilkan dengan warna biru muda, node yang telah dikunjungi ditampilkan dengan warna hijau, sedangkan node yang termasuk ke dalam jalur hasil pencarian ditampilkan dengan warna oranye. Dengan adanya visualisasi ini, pengguna dapat lebih mudah memahami proses pencarian yang dilakukan oleh algoritma BFS maupun DFS.

Selain itu, program menyediakan tombol Reset yang berfungsi untuk menghapus hasil pencarian sebelumnya, mengembalikan warna node ke kondisi awal, serta mengosongkan data jalur dan node yang telah dikunjungi sehingga pengguna dapat melakukan pencarian baru dengan lokasi yang berbeda.

Secara keseluruhan, program ini berhasil mengimplementasikan konsep graph, algoritma BFS, dan algoritma DFS dalam sebuah aplikasi GUI Java. Program mampu menampilkan visualisasi graph, menentukan lokasi awal dan tujuan, mencari jalur menggunakan BFS maupun DFS, menampilkan jalur yang ditemukan, menampilkan node yang dikunjungi, serta menghitung jumlah node yang dieksplorasi sesuai dengan ketentuan tugas yang diberikan

.

.